

Lab overview

In this lab, I'll practice troubleshooting connectivity to resources in a Virtual Private Cloud (VPC). VPC is a virtual network that I define and control. It allows me to launch AWS resources, such as EC2 instances, databases, and other services, in a logically isolated virtual network. A security group is a virtual firewall that controls inbound and outbound traffic to resources within a cloud computing environment. Security groups are a common source of problems with connectivity to resources in a VPC. To practice troubleshooting this type of issue for connectivity, I'll first review what traffic should be allowed through security groups. I'll then develop connectivity tests to identify misconfigured security group rules. Lastly, I'll remediate connectivity through security groups by editing the access rules.

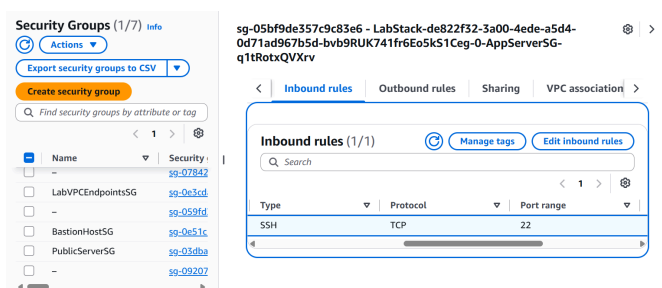
Objectives

In this lab, I'll do the following:

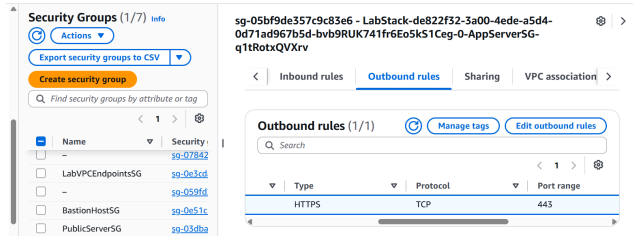
- Examine security groups and determine what traffic is allowed.
- Change which security groups are applied to EC2 instances.
- Update security groups to follow the principle of least privilege.
- Understand how security groups can reference other security groups.

Task 1: Inspecting the App Server instance security group

In this task, I'll review the inbound and outbound security group rules attached to the App Server instance. The current *AnyCompany* security policy states that all SSH traffic to instances in private subnets must connect through a bastion host. I'll inspect the AppServerSG security group rules to see what inbound traffic is currently allowed into the App Server instance.



This configuration allows inbound connections to the App Server from *any* IP address over port 22 (this is the SSH port). All other ports are denied access. This allows the *Bastion Host* to connect to the *App Server*. However, unauthorized hosts from *any* IP address (source 0.0.0.0/0) can also connect to the *App Server*. Currently, this security group is *noncompliant* based on the AnyCompany's security policy.



The current rules allow HTTPS (port 443) outbound traffic from the EC2 instance to the VPC endpoints security group only. This restricts the App Server to communicating with AWS services (such as Systems Manager) within the VPC, while blocking all other outbound traffic.

I successfully reviewed the *inbound* and *outbound* security group rules for the *App Server* instance.

Task 2: Developing tests for SSH connectivity to the App Server instance

In this task, I'll test the current App Server security group rules by attempting to connect using SSH to the App Server instance from the Bastion Host and the Public Server instances.

To display the **hostname** (which includes the private IPv4 address), **instance ID**, and **instance tag name** of the instance that I am currently connected to, I'll run the following commands:

```
sh-5.2$ # Get the token to retrieve UserData
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" 2>/dev/null)

# Get the instance ID and set to a variable
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http:// 169.254.169.254/latest/meta-data/instance-id 2>/dev/null)
if [ -z "$INSTANCE_ID" ]; then
    echo "Error: Unable to retrieve the instance ID." >&2
fi

# Get the tag value for "Name" and set to a variable
TAG_NAME=$(aws ec2 describe-tags --filters "Name=resource-id, Values=$INSTANCE_ID" "Name=key,Values=Name" --query "Tags[0].Value" --output text 2>/dev/null)
if [ -z "$TAG_NAME" ]; then
    echo "Error: Unable to retrieve the tag value for 'Name'." >&2
fi

# Get the hostname and set to a variable
HNAME=$(hostname)
# Output the results
clear
echo -e "\n\nHost Name: $HNAME"
echo "Instance ID: $INSTANCE_ID"
echo "Instance Tag Name: $TAG_NAME"
```

The above command gives the following output;

```
Host Name: ip-10-10-10-89.us-west-2.compute.internal
Instance ID: i-094b774ceca8a60af
Instance Tag Name: Bastion Host
sh-5.2$
```

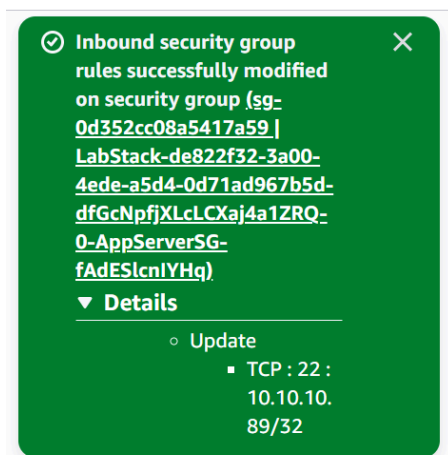
To start the already installed **ssh-agent** on the **Bastion Host**, I'll run the following command to give the following output;

This command displays the hostname of the instance that I am currently connected to (App Server), which includes the private IPv4 address assigned to App Server. This confirms that I have successfully opened an SSH session to App Server from Public Server. *This is noncompliant behavior and must be corrected.*

I successfully tested the current App Server security group rules by attempting to connect by using SSH into the App Server instance from the Bastion Host and the Public Server instances.

Task 3: Restricting SSH access to App Server from a specific IPv4 address

In this task, I'll update the AppServerSG to allow SSH access, specifying the internal IPv4 address of Bastion Host as the only possible source. To make the App Server security group compliant, I'll modify the rules to allow incoming SSH traffic from the Bastion Host.



I have now updated the security group attached to App Server so that it *only* allows SSH traffic from Bastion Host. Now it's time to test my changes for compliance.

To verify the **hostname**, **instance ID**, and **instance tag name**, I'll run the following commands:

```
sh-5.2$ # Get the token to retrieve UserData
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" 2>/dev/null)

# Get the instance ID and set to a variable
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http:// 169.254.169.254/latest/meta-data/instance-id 2>/dev/null)
if [ -z "$INSTANCE_ID" ]; then
    echo "Error: Unable to retrieve the instance ID." >&2
fi

# Get the tag value for "Name" and set to a variable
TAG_NAME=$(aws ec2 describe-tags --filters "Name=resource-id, Values=$INSTANCE_ID" "Name=key,Values=Name" --query "Tags[0].Value" --output text 2>/dev/null)
if [ -z "$TAG_NAME" ]; then
    echo "Error: Unable to retrieve the tag value for 'Name'." >&2
fi

# Get the hostname and set to a variable
HNAME=$(hostname)
# Output the results
clear
echo -e "\n\nHost Name: $HNAME"
echo "Instance ID: $INSTANCE_ID"
echo "Instance Tag Name: $TAG_NAME"
```

The above command gives the following output;

```
Host Name: ip-10-10-10-138.us-west-2.compute.internal
Instance ID: i-06663c7b4483cbcbd
Instance Tag Name: Public Server
sh-5.2$
```

To connect to the **APP Server** instance, in the **Public Server** browser tab terminal, I'll run the following command which will give the following expected output;

```
sh-5.2$ ssh -A ec2-user@10.10.10.126
ssh: connect to host 10.10.10.126 port 22: Connection timed out
sh-5.2$
```

The command pauses and does not complete. After a few minutes, I receive a message stating that the connection timed out. This confirms that I was unsuccessful in opening an SSH session to the *App Server* from *Public Server*. This is compliant behavior. Next, I'll confirm SSH connectivity to the *App Server* from the *Bastion Host*.

To verify the **hostname**, **instance ID**, and **instance tag name**, I'll run the following commands:

```
sh-5.2$ # Get the token to retrieve UserData
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" 2>/dev/null)

# Get the instance ID and set to a variable
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http:// 169.254.169.254/latest/meta-data/instance-id 2>/dev/null)
if [ -z "$INSTANCE_ID" ]; then
    echo "Error: Unable to retrieve the instance ID." >&2
fi

# Get the tag value for "Name" and set to a variable
TAG_NAME=$(aws ec2 describe-tags --filters "Name=resource-id, Values=$INSTANCE_ID" "Name=key,Values=Name" --query "Tags[0].Value" --output text 2>/dev/null)
if [ -z "$TAG_NAME" ]; then
    echo "Error: Unable to retrieve the tag value for 'Name'." >&2
fi

# Get the hostname and set to a variable
HNAME=$(hostname)
# Output the results
clear
echo -e "\n\nHost Name: $HNAME"
echo "Instance ID: $INSTANCE_ID"
echo "Instance Tag Name: $TAG_NAME"
```

The above command gives the following output;

```
Host Name: ip-10-10-10-89.us-west-2.compute.internal
Instance ID: i-094b774ceca8a60af
Instance Tag Name: Bastion Host
sh-5.2$
```

To connect to the App Server instance from the Bastion Host browser terminal, I'll run the following command, which will give the expected output;

```
Host Name: ip-10-10-10-89.us-west-2.compute.internal
Instance ID: i-094b774ceca8a60af
Instance Tag Name: Bastion Host
sh-5.2$ ssh -A ec2-user@10.10.10.126
ec2-user@10.10.10.126's password:

#_
~\##### Amazon Linux 2023
~~\#####\
~~\###|
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~V~' '->
~~~
~~~.
~/m/'
```

Image depicts the ssh login boilerplate when connecting to the APP Server. I am now connected to the App Server.

To see the **hostname**, **instance ID**, and **instance tag name**, I'll run the following commands:

```
[ec2-user@ip-10-10-10-126 ~]$ # Get the token to retrieve UserData
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" 2>/dev/null)

# Get the instance ID and set to a variable
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http:// 169.254.169.254/latest/meta-data/instance-id 2>/dev/null)
if [ -z "$INSTANCE_ID" ]; then
    echo "Error: Unable to retrieve the instance ID." >&2
fi

# Get the tag value for "Name" and set to a variable
TAG_NAME=$(aws ec2 describe-tags --filters "Name=resource-id, Values=$INSTANCE_ID" "Name=key,Values=Name" --query "Tags[0].Value" --output text 2>/dev/null)
if [ -z "$TAG_NAME" ]; then
    echo "Error: Unable to retrieve the tag value for 'Name'." >&2
fi

# Get the hostname and set to a variable
HNAME=$(hostname)
# Output the results
clear
echo -e "\n\nHost Name: $HNAME"
echo "Instance ID: $INSTANCE_ID"
echo "Instance Tag Name: $TAG_NAME"
```

The above command gives the following output;

```
Host Name: ip-10-10-10-126.us-west-2.compute.internal
Instance ID: i-03e971aa6057bac80
Instance Tag Name: App Server
[ec2-user@ip-10-10-10-126 ~]$
```

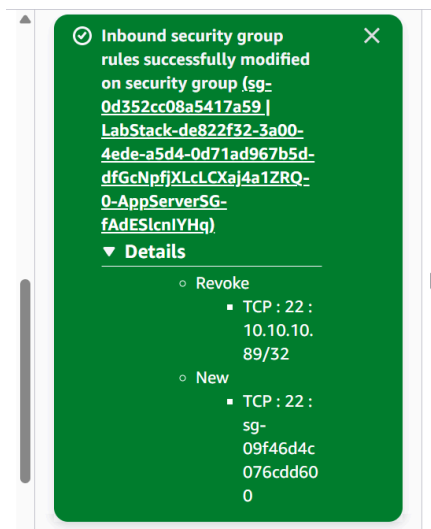
This confirms that I have successfully opened an *SSH session* to *App Server* from the *Bastion Host*. This is compliant behavior.

I successfully updated the AppServerSG to allow SSH access, specifying the internal IPv4 address of Bastion Host as the only possible source. To make the App Server security group compliant, I modified the rules to allow incoming SSH traffic from the Bastion Host.

Task 4: Referencing security groups as the inbound source for access control

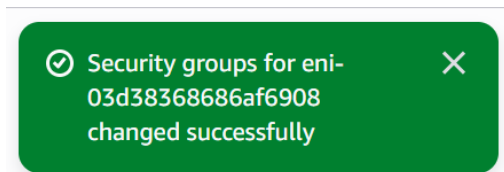
In the previous task, I modified the App Server security group rule to allow inbound SSH connections from the Bastion Host instance. In this task, I'll update the App Server security group to allow any instances assigned with the BastionHostSG security group to send SSH traffic to the App Server. This is useful when multiple servers need to have the same type of connectivity to a resource.

For this example, I'll turn the Public Server into another Bastion Host, and add it to the BastionHostSG security group.



I have now updated the *security group* attached to *App Server* so that it *only* allows traffic from instances with the *BastionHostSG* security group attached.

Next, I'll change the security group assigned to Public Server to be the same as Bastion Host.



Now it's time to test the connectivity from the redundant bastion host (Public Server).

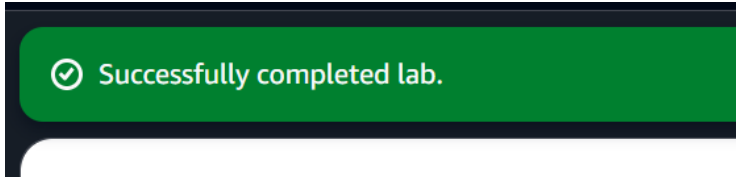
To connect to the **App Server** from the the **PublicServerSessionManagementUrl** browser tab terminal, I'll run the following command:

I successfully updated the App Server security group to allow any instances with BastionHostSG to send SSH traffic to the App Server.

Conclusion

I have successfully:

- Examined security groups and determined what traffic is allowed.
- Changed which security groups are applied to Amazon EC2 instances.
- Updated security groups to follow the principle of least privilege.
- Understood how security groups can reference other security groups.

A green rectangular box with rounded corners and a dark green border. It contains a white checkmark icon followed by the text "Successfully completed lab." in white.

✓ Successfully completed lab.